

Wits Quest

Performance and Test Coverage Report

Milestone 4 · measured 9 October 2026 and 10 October 2026 · code at commit 82d10ec

2,976

API requests in the load test, 0 failed

92–100

Lighthouse accessibility, 19 screens
and themes

49 / 67

Lighthouse performance on a phone:
Login / Map

946

automated tests, all passing (628
backend, 318 frontend)

81.75%

backend line coverage

92.18% / 52.34%

frontend line coverage: the gated files
/ all code

This report records how Wits Quest was tested for speed, load, accessibility and coverage, with the results and screenshots as evidence. Every number comes from the files the test scripts saved (`scripts/performance/`); the PDF itself is built by `npm run report`.

Live app: wits-quest.vercel.app · Live API: wits-quest.onrender.com · Browser: Chrome 154.0.8037.98 · Lighthouse 12.2.1 · autocannon 7.15.0 · Vitest 1.6 with V8 coverage

1. API load test

What: eight runs against the live API, each a number of simultaneous users sending requests back to back.
Read-only: GET requests only, plus one sign-in for the signed-in endpoint (as the seeded test player 2999001).
10 users for 20 seconds per endpoint, then two 30-user bursts of 15 seconds. Run on Fri, 09 Oct 2026 21:55:30 GMT.

Endpoint	Users	Requests	Per second	Median	97.5%	99%	Slowest	Failed
Health check <code>/api/health</code>	10	529	26.5	350 ms	518 ms	1035 ms	1130 ms	0
Card catalogue <code>/api/cards</code>	10	317	15.9	587 ms	915 ms	1176 ms	1336 ms	0
Campus events <code>/api/events</code>	10	321	16.1	593 ms	865 ms	927 ms	1003 ms	0
Leaderboard <code>/api/users/leaderboard</code>	10	317	15.9	606 ms	821 ms	1042 ms	1357 ms	0
Territories <code>/api/territories</code>	10	153	7.7	1203 ms	1906 ms	1931 ms	1985 ms	0
Tournaments (signed in) <code>/api/tournaments</code>	10	142	7.1	1320 ms	1697 ms	1703 ms	1721 ms	0
Burst: card catalogue <code>/api/cards</code>	30	598	39.9	714 ms	1157 ms	1311 ms	1736 ms	0
Burst: leaderboard <code>/api/users/leaderboard</code>	30	599	39.9	707 ms	1109 ms	1296 ms	1457 ms	0

- **No failures:** 2,976 requests, every one answered with success. No errors, no time-outs, and the server didn't fall over under 30 users at once.
- **Typical response:** about 0.35 s for the health check and 0.6 s for the card catalogue, events and leaderboard. Territories and the signed-in tournament list are the slowest, at about 1.2–1.3 s: each answers with several database reads.
- **Much of every response is distance.** Requests reach Cloudflare in Johannesburg in about 23 ms, but the server is on Render's free plan in its default region (Oregon, US): the first byte of even the health check takes about 0.5 s. Moving the service to a region nearer South Africa would cut every response.
- **Under a burst** of 30 users, throughput rose to about 40 requests a second, and the median only went from about 0.6 s to 0.7 s.

First request

A single health check on a fresh connection took **596 ms** (HTTP 200) while the server was awake. Render's free plan puts the server to sleep after about 15 minutes without traffic; the first request after that waits while it starts again, which can take up to a minute. That wait wasn't measured here (the server was in use), so keeping it awake during marking (a scheduled ping to `/api/health`) is the fix to rely on.

Pretty-print ☐

```
{"status":"ok","service":"Wits Quest API (Supabase)","timestamp":"2026-10-09T22:16:40.134Z","commit":"99999e4b0bf3efff954d82da4f82d7a9abb43f3a"}
```

The live API's health check when this report was made, showing the deployed commit (99999e4).

Raw results (results/load-test.json)

```
{
  "base": "https://wits-quest.onrender.com",
  "ranAt": "2026-10-09T21:55:30.702Z",
  "firstRequest": {
    "status": 200,
    "ms": 596
  },
  "results": [
    {
      "name": "Health check",
      "path": "/api/health",
      "users": 10,
      "seconds": 20,
      "requests": 529,
      "perSecond": 26.5,
      "medianMs": 350,
      "p97_5Ms": 518,
      "p99Ms": 1035,
      "maxMs": 1130,
      "ok": 529,
      "notOk": 0,
      "errors": 0,
      "timeouts": 0
    },
    {
      "name": "Card catalogue",
      "path": "/api/cards",
      "users": 10,
      "seconds": 20,
      "requests": 317,
      "perSecond": 15.9,
      "medianMs": 587,
      "p97_5Ms": 915,
      "p99Ms": 1176,
      "maxMs": 1336,
      "ok": 317,
      "notOk": 0,
      "errors": 0,
      "timeouts": 0
    },
    {
      "name": "Campus events",
      "path": "/api/events",
      "users": 10,
      "seconds": 20,
      "requests": 321,
      "perSecond": 16.1,
      "medianMs": 593,
      "p97_5Ms": 865,
      "p99Ms": 927,
      "maxMs": 1003,
      "ok": 321,
      "notOk": 0,
      "errors": 0,
      "timeouts": 0
    }
  ]
}
... (5 more runs in the file)
```

2. Lighthouse: speed, accessibility, best practices, SEO

What: Lighthouse 12 on the live app, with its phone profile (a mid-range phone on slow 4G), signed in as the test player. Login and the Map (the first screen after signing in) are full page loads, so they get a speed score. The other screens are opened from the bottom bar, as a player would, and checked as they stand: Lighthouse can't score the speed of a screen that isn't a page load. Map, Cards and Battle are checked again in all 4 themes, for colour contrast. Run on Fri, 09 Oct 2026 22:07:11 GMT.

Screen	How	Performance	Accessibility	Best practices	SEO
Login (first visit)	page load	49	100	100	90
Map (signed in)	page load	67	92	96	91
Cards	opened from the bar	—	100	100	80
Quests	opened from the bar	—	100	100	75
Battle	opened from the bar	—	100	100	75
Ranks	opened from the bar	—	100	100	75
Me	opened from the bar	—	100	88	75
Map, night theme	opened from the bar	—	96	88	80
Cards, night theme	opened from the bar	—	100	100	80
Battle, night theme	opened from the bar	—	100	100	75
Map, day theme	opened from the bar	—	96	88	80
Cards, day theme	opened from the bar	—	100	100	80
Battle, day theme	opened from the bar	—	100	100	75
Map, blue theme	opened from the bar	—	96	88	80
Cards, blue theme	opened from the bar	—	100	100	80
Battle, blue theme	opened from the bar	—	100	100	75
Map, pink theme	opened from the bar	—	96	88	80
Cards, pink theme	opened from the bar	—	100	100	80
Battle, pink theme	opened from the bar	—	100	100	75

Speed on a phone

Page load	First paint	Largest paint	Blocking time	Layout shift	Speed Index
Login (first visit)	10.0 s	10.5 s	320 ms	0.001	10.0 s
Map (signed in)	1.0 s	3.1 s	1,390 ms	0	3.6 s

Why the first visit is slow: the whole app is one 1.55 MB JavaScript file (431 KB compressed): every screen, the admin console, the map library and the QR scanner. A phone on slow 4G downloads and runs all of it, and waits for the Google Fonts stylesheet, before the login page appears. Loading each screen only when it's opened (code splitting) would cut the first visit the most. The Map's 1.4 s of blocking time is the map library and its markers starting up.

Lighthouse summary of every step

Lighthouse User Flow Report
Save
? Understanding Flows

Wits Quest on a phone

Oct 10, 2026, 12:04 AM GMT+2

- Summary

Emulated Moto G Power - 412x823, DPR 1.75
 Slow 4G throttling
 4x slowdown

Summary

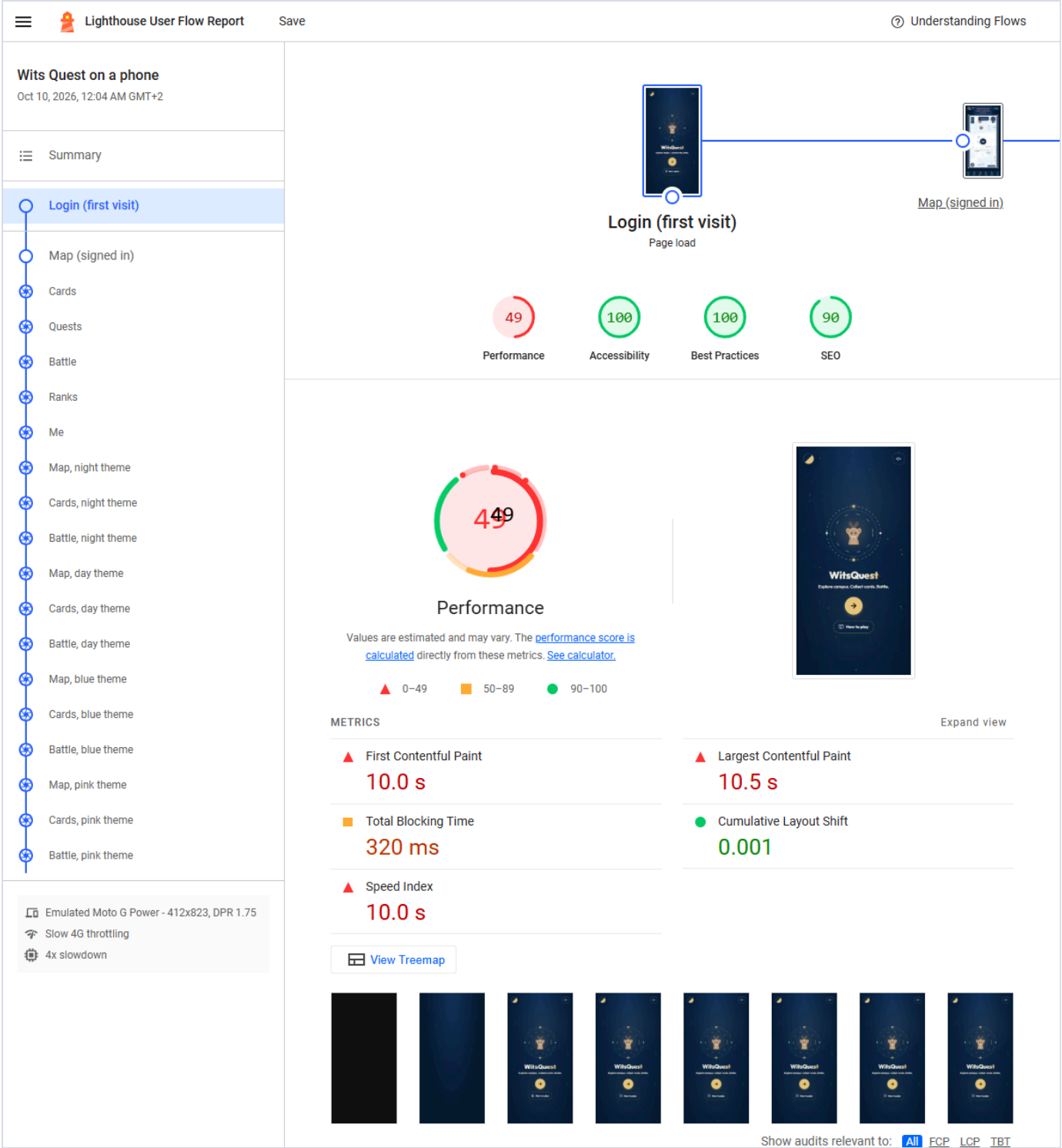
2 navigation reports · 17 snapshot reports

ALL REPORTS

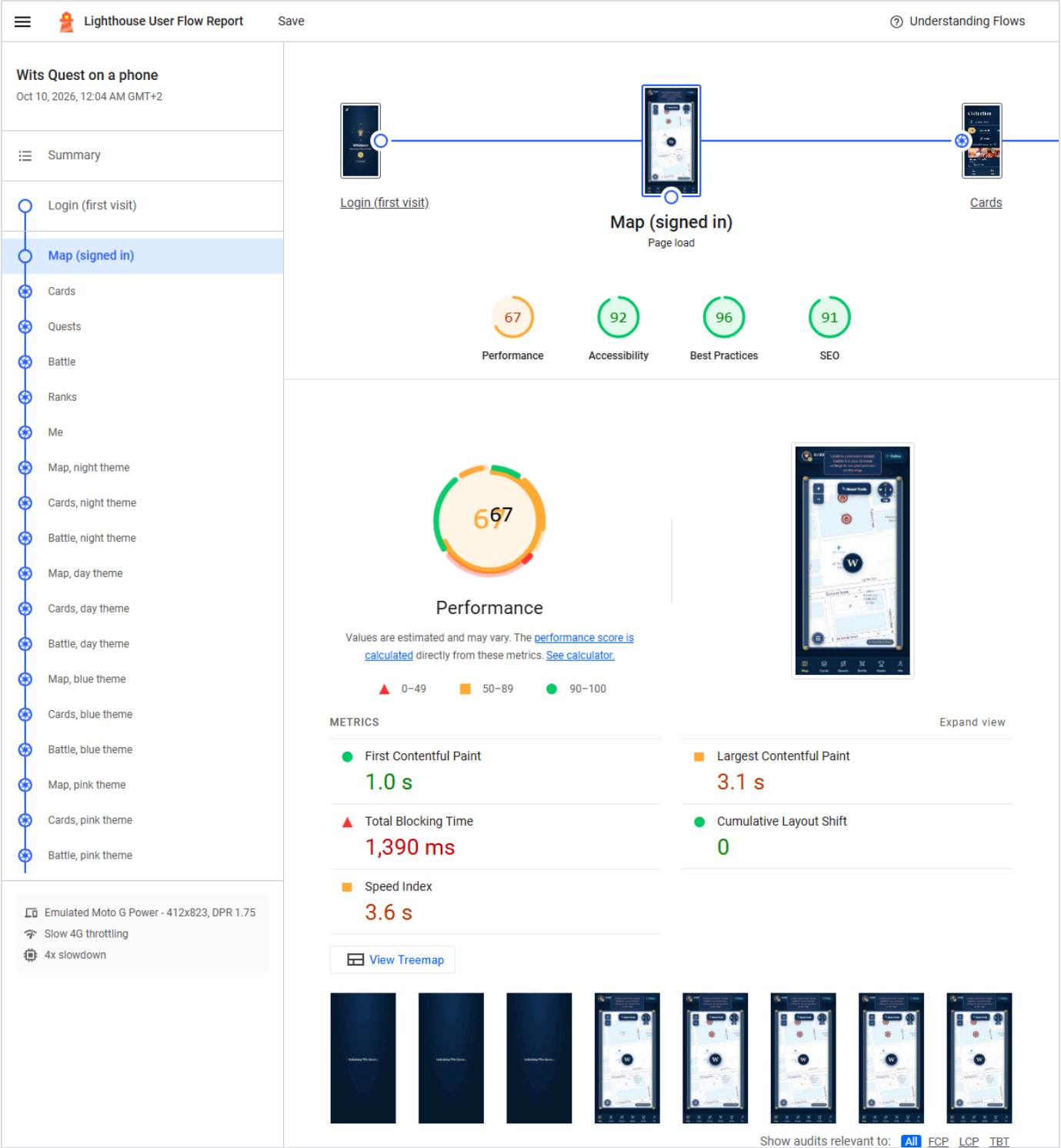
	Performance	Accessibility	Best Practices	SEO
https://wits-quest.vercel.app/				
Page load				
Login (first visit)	49	100	100	90
https://wits-quest.vercel.app/				
Page load				
Map (signed in)	67	92	96	91
Captured state of page				
Cards	2/3	24/24	6/6	4/5
Captured state of page				
Quests	3/3	20/20	6/6	3/4
Captured state of page				
Battle	3/3	14/14	6/6	3/4
Captured state of page				
Ranks	3/3	20/20	6/6	3/4
Captured state of page				
Me	3/3	20/20	5/6	3/4
Captured state of page				
Map, night theme	3/3	23/24	5/6	4/5
Captured state of page				
Cards, night theme	2/3	24/24	6/6	4/5
Captured state of page				
Battle, night theme	3/3	14/14	6/6	3/4
Captured state of page				
Map, day theme	3/3	23/24	5/6	4/5

The Lighthouse report's summary: every step of the run with its scores.

Login, first visit (page load)



Map, signed in (page load)



Map: the accessibility checks that failed

Lighthouse User Flow Report

Save

Understanding Flows

Wits Quest on a phone

Oct 10, 2026, 12:04 AM GMT+2

Summary

Login (first visit)

Map (signed in)

Cards

Quests

Battle

Ranks

Me

Map, night theme

Cards, night theme

Battle, night theme

Map, day theme

Cards, day theme

Battle, day theme

Map, blue theme

Cards, blue theme

Battle, blue theme

Map, pink theme

Cards, pink theme

Battle, pink theme

Emulated Moto G Power - 412x823, DPR 1.75

Slow 4G throttling

4x slowdown

92

Accessibility

These checks highlight opportunities to [improve the accessibility of your web app](#). Automatic detection can only detect a subset of issues and does not guarantee the accessibility of your web app, so [manual testing](#) is also encouraged.

ARIA

▲ button, link, and menuitem elements do not have accessible names.

These are opportunities to improve the usage of ARIA in your application which may enhance the experience for users of assistive technology, like a screen reader.

BEST PRACTICES

▲ Touch targets do not have sufficient size or spacing.

These items highlight common accessibility best practices.

NAVIGATION

○ The page contains a heading, skip link, or landmark region

These are opportunities to improve keyboard navigation in your application.

ADDITIONAL ITEMS TO MANUALLY CHECK (10)

These items address areas which an automated testing tool cannot cover. Learn more in our guide on [conducting an accessibility review](#).

PASSED AUDITS (22)

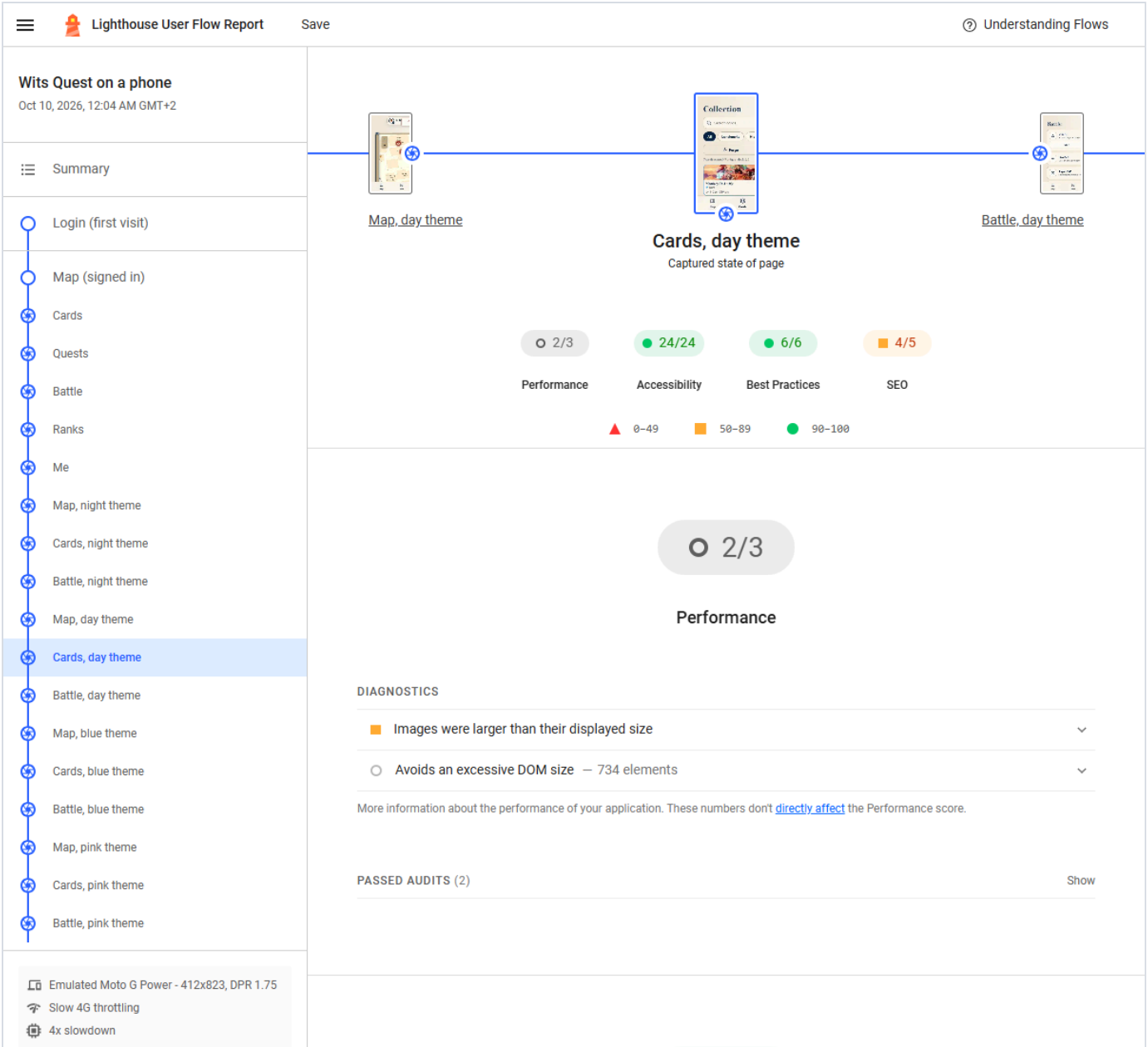
NOT APPLICABLE (32)

The Map's accessibility section, with the checks that failed.

Wits Quest · Performance and Test Coverage Report

Page 10 of 20

Cards in the Daylight theme (opened from the bar)



Every check that failed, and the fix

Category	Check	Where	What's wrong
best-practices	Missing source maps for large first-party JavaScript	Login (first visit), Map (signed in)	The production build ships without source maps, so errors in the minified code are harder to trace.
seo	Document does not have a meta description	every screen	index.html has no description for search results.
accessibility	`button`, `link`, and `menuitem` elements do not have accessible names.	5 of 19 screens	The event markers on the map can be tapped but have no name for screen readers (Leaflet markers drawn as plain elements).
accessibility	Touch targets do not have sufficient size or spacing.	Map (signed in)	The live player marker on the map is a small tap target.
accessibility	Elements with visible text labels do not have matching accessible names.	Map (signed in), Me	The compass button reads "Turn on the live compass" while it shows "Tap"; the display-name button on Me reads "Edit display name" while it shows the name.
best-practices	Serves images with low resolution	Map (signed in)	OpenStreetMap tiles are served at one size, so phones with dense screens get them stretched.
best-practices	Document doesn't use legible font sizes	5 of 19 screens	Some text is under 12 px: on the Map (the folding map credit, at 10 px) and on Me.

Accessibility in short: 100 on Login, Cards, Quests, Battle, Ranks and Me, and in all 4 themes on Cards and Battle: text contrast passes everywhere. The Map scores 92–96 because of its markers and the compass button's label.

3. Automated tests and coverage

What: both test suites, run on the code at commit 82d10ec, with V8 coverage. They never touch the live database: the backend's tests use an in-memory stand-in or mocks for Supabase, and the frontend's mock the API. How the tests are built is in *How the Tests Are Built* on the docs site.

Suite	Test files	Tests	Result
Backend (Vitest, Supertest, real Socket.IO clients)	44	628	all passed
Frontend (Vitest, React Testing Library, jsdom)	44	318	all passed

Measured over	Lines	Statements	Functions	Branches	Lines covered	Notes
Backend	81.75%	81.75%	92.27%	76.98%	12,670 of 15,497	Seed data and stale report files excluded (see below)
Frontend, the gated files	92.18%	92.18%	82.22%	82.19%	1,770 of 1,920	The 9 files the frontend gate lists
Frontend, all code	52.34%	52.34%	52.32%	76.92%	19,905 of 38,027	Every source file except tests and previews

Two things to read the numbers by. (1) The backend figure excludes `src/db/productionContent.ts` and `seedProduction.ts` (the production seed data and its loader, about 860 lines) and any old HTML report lying in `backend/coverage/`, in line with `seed.ts` and `src/scripts/`, which were already excluded. Counting them, it was 74%. (2) The frontend gate measures 9 files (the battle engine, the CPU opponent, the deck and card services and three components). Across all of the frontend's code, coverage is 52.34%: the map and admin screens are the least tested.

Least-covered files (100 lines or more)

Backend file	Lines covered	Frontend file	Lines covered
/src/routes/adminAchievements.ts	31.0% of 171	/src/components/BattleHistoryModal.tsx	0.0% of 270
/src/routes/telemetry.ts	31.1% of 1032	/src/components/StudentOpponentDrawer.tsx	0.0% of 571
/src/routes/battle.ts	37.3% of 351	/src/components/WitsLogo.tsx	0.0% of 150
/src/routes/ranked.ts	60.5% of 152	/src/screens/QuestTrails.tsx	5.9% of 306
/src/utils/achievements.ts	62.0% of 221	/src/screens/Appearance.tsx	6.1% of 179
/src/routes/asyncBattle.ts	65.3% of 949	/src/screens/admin/AdminContent.tsx	6.4% of 2026

Test run output

BACKEND

Test Files 44 passed (44)
Tests 628 passed (628)
Start at 00:08:13
Duration 17.33s (transform 6.91s, setup 17ms, collect 43.92s, tests 21.27s, environment 32ms, prepare 24.23s)

===== Coverage summary =====

Statements : 81.75% (12670/15497)
Branches : 76.98% (2238/2907)
Functions : 92.27% (239/259)
Lines : 81.75% (12670/15497)

FRONTEND (the gated files)

Test Files 44 passed (44)
Tests 318 passed (318)
Start at 00:08:36
Duration 39.08s (transform 7.50s, setup 24.23s, collect 31.01s, tests 43.15s, environment 96.70s, prepare 24.68s)

===== Coverage summary =====

Statements : 92.18% (1770/1920)
Branches : 82.19% (217/264)
Functions : 82.22% (37/45)
Lines : 92.18% (1770/1920)

FRONTEND (all code)

Test Files 44 passed (44)
Tests 318 passed (318)
Start at 00:09:21
Duration 38.13s (transform 7.54s, setup 24.16s, collect 30.35s, tests 41.54s, environment 95.93s, prepare 23.99s)

===== Coverage summary =====

Statements : 52.34% (19905/38027)
Branches : 76.92% (2257/2934)
Functions : 52.32% (451/862)
Lines : 52.34% (19905/38027)

Backend coverage report

All files

81.75% Statements

12670/15497

76.98% Branches

2238/2907

92.27% Functions

239/259

81.75% Lines

12670/15497

Press *n* or *j* to go to the next uncovered block, *b*, *p* or *k* for the previous block.

Filter:

File		Statements		Branches		Functions		Lines	
src		72.72%	352/484	65.57%	40/61	75%	3/4	72.72%	352/484
src/config		100%	19/19	100%	0/0	100%	0/0	100%	19/19
src/db		83.33%	15/18	0%	0/1	100%	0/0	83.33%	15/18
src/docs		100%	1005/1005	100%	15/15	100%	1/1	100%	1005/1005
src/middleware		96.09%	271/282	89.83%	53/59	100%	8/8	96.09%	271/282
src/routes		71.92%	5774/8028	73.56%	1035/1407	84.05%	58/69	71.92%	5774/8028
src/services		91.84%	3526/3839	77.18%	751/973	95.37%	103/108	91.84%	3526/3839
src/utills		93.74%	1708/1822	87.97%	344/391	95.65%	66/69	93.74%	1708/1822

Code coverage generated by [istanbul](#) at 2026-10-09T22:08:32.175Z

The backend's coverage report (V8), by folder.

Backend coverage, file by file: the routes

All files src/routes

71.92% Statements

5774/8028

73.56% Branches

1035/1407

84.05% Functions

58/69

71.92% Lines

5774/8028

Press *n* or *j* to go to the next uncovered block, *b*, *p* or *k* for the previous block.

Filter:

File		Statements		Branches		Functions		Lines	
adminAchievements.ts		30.99%	53/171	100%	0/0	0%	0/1	30.99%	53/171
analyticsQuestions.ts		41.66%	20/48	100%	0/0	100%	0/0	41.66%	20/48
asyncBattle.ts		65.33%	620/949	63.97%	87/136	66.66%	4/6	65.33%	620/949
auth.ts		70.71%	425/601	82.79%	77/93	50%	1/2	70.71%	425/601
battle.ts		37.32%	131/351	75%	6/8	0%	0/3	37.32%	131/351
content.ts		84.35%	1208/1432	78.93%	266/337	100%	2/2	84.35%	1208/1432
docs.ts		100%	83/83	71.42%	10/14	100%	0/0	100%	83/83
forge.ts		100%	91/91	70%	14/20	100%	0/0	100%	91/91
friends.ts		96.76%	509/526	73.12%	117/160	100%	12/12	96.76%	509/526
qrCheckin.ts		92.66%	316/341	72.72%	40/55	100%	5/5	92.66%	316/341
ranked.ts		60.52%	92/152	53.84%	7/13	100%	0/0	60.52%	92/152
routing.ts		93.35%	239/256	86.04%	37/43	100%	5/5	93.35%	239/256
starterPack.ts		84.39%	119/141	58.82%	20/34	66.66%	2/3	84.39%	119/141
telemetry.ts		31.1%	321/1032	94.23%	49/52	57.14%	4/7	31.1%	321/1032
territory.ts		93.77%	437/466	71.55%	83/116	100%	10/10	93.77%	437/466
tournaments.ts		95.51%	149/156	80.48%	33/41	100%	2/2	95.51%	149/156
trades.ts		91.13%	545/598	71.72%	137/191	100%	11/11	91.13%	545/598
trails.ts		65.61%	416/634	55.31%	52/94	100%	0/0	65.61%	416/634

Code coverage generated by [istanbul](#) at 2026-10-09T22:08:32.175Z

The backend's routes, file by file: telemetry.ts, battle.ts and adminAchievements.ts are the gaps.

Frontend coverage report: the gated files

All files

92.18% Statements

1778/1928

82.19% Branches

217/264

82.22% Functions

37/45

92.18% Lines

1778/1928

Press *n* or *j* to go to the next uncovered block, *b*, *p* or *k* for the previous block.

Filter:

File		Statements		Branches		Functions		Lines	
components		87.76%	610/695	68.75%	44/64	50%	5/10	87.76%	610/695
screens		97.08%	133/137	80%	36/45	66.66%	2/3	97.08%	133/137
services		98.34%	297/302	67.74%	21/31	100%	7/7	98.34%	297/302
utils		92.87%	730/786	93.54%	116/124	92%	23/25	92.87%	730/786

Code coverage generated by [istanbul](#) at 2026-10-09T22:09:17.356Z

The frontend's coverage report over the 9 files its gate measures.

Frontend coverage report: all code

All files

52.34% Statements

19905/38027

76.92% Branches

2257/2934

52.32% Functions

451/862

52.34% Lines

19905/38027

Press *n* or *j* to go to the next uncovered block, *b*, *p* or *k* for the previous block.

Filter:

File		Statements		Branches		Functions		Lines	
src		10.76%	89/827	0%	0/1	0%	0/3	10.76%	89/827
src/components		54.45%	2293/4211	81.49%	251/308	60.46%	52/86	54.45%	2293/4211
src/components/admin		17.22%	51/296	100%	0/0	0%	0/3	17.22%	51/296
src/components/auth		29.33%	120/409	100%	0/0	0%	0/9	29.33%	120/409
src/components/battle		100%	341/341	100%	42/42	100%	2/2	100%	341/341
src/components/ui		79.15%	676/854	65.88%	56/85	77.77%	14/18	79.15%	676/854
src/context		85.43%	352/412	43.66%	31/71	70.58%	12/17	85.43%	352/412
src/engine		0%	0/35	0%	0/1	0%	0/1	0%	0/35
src/screens		51.74%	8412/16257	73.37%	1047/1427	53.46%	139/260	51.74%	8412/16257
src/screens/admin		37.72%	3250/8616	84.51%	382/452	49.36%	78/158	37.72%	3250/8616
src/services		74.21%	3264/4398	77.57%	301/388	47.28%	113/239	74.21%	3264/4398
src/styles		0%	0/82	0%	0/1	0%	0/1	0%	0/82
src/theme		36.84%	42/114	100%	0/0	0%	0/6	36.84%	42/114
src/utlis		86.38%	1015/1175	93.03%	147/158	69.49%	41/59	86.38%	1015/1175

Code coverage generated by [istanbul](#) at 2026-10-09T22:10:04.173Z

Frontend coverage, file by file: the screens

All files src/screens

51.74% Statements8412/1625773.37% Branches1047/142753.46% Functions139/26851.74% Lines8412/16257

Press *n* or *j* to go to the next uncovered block, *b*, *p* or *k* for the previous block.

Filter:

File	Statements	Branches	Functions	Lines
Appearance.tsx	6.14%11/179	100%0/0	0%0/3	6.14%11/179
AsyncBattleArena.tsx	66.33%806/1215	47.74%74/155	42.85%3/7	66.33%806/1215
AsyncPvP.tsx	66.85%359/537	60%36/60	33.33%4/12	66.85%359/537
BattleArena.tsx	46.52%750/1612	47.36%27/57	17.85%5/28	46.52%750/1612
BattleHistory.tsx	11.51%16/139	100%0/0	0%0/1	11.51%16/139
CardCollection.tsx	44.95%312/694	79.36%50/63	27.77%5/18	44.95%312/694
CardForge.tsx	97.37%260/267	79.62%43/54	83.33%5/6	97.37%260/267
EmailVerification.tsx	12.19%20/164	100%0/0	0%0/2	12.19%20/164
Friends.tsx	99.83%606/607	91%91/100	87.5%21/24	99.83%606/607
HowToPlay.tsx	100%131/131	100%11/11	100%3/3	100%131/131
Leaderboard.tsx	96.62%774/801	84.4%92/109	84.61%11/13	96.62%774/801
LivePvPArena.tsx	38.27%640/1672	42.1%32/76	15.38%2/13	38.27%640/1672
Login.tsx	43.34%319/736	63.63%7/11	28.57%4/14	43.34%319/736
MapExplorer.tsx	6.74%205/3041	0%0/1	0%0/18	6.74%205/3041
Onboarding.tsx	22.46%31/138	100%0/0	0%0/3	22.46%31/138
Profile.tsx	7.83%34/434	100%0/0	0%0/1	7.83%34/434
QuestTrails.tsx	5.88%18/306	100%0/0	0%0/1	5.88%18/306
RankedMatchmaking.tsx	97.08%133/137	80%36/45	66.66%2/3	97.08%133/137
SpectateArena.tsx	95.57%713/746	81.64%129/158	88.88%16/18	95.57%713/746
TerritoryMap.tsx	7.46%20/268	100%0/0	0%0/1	7.46%20/268
Tournaments.tsx	96.24%1077/1119	79.02%211/267	83.33%30/36	96.24%1077/1119
Trades.tsx	93.58%729/779	83.33%125/150	85.18%23/27	93.58%729/779
TriviaModal.tsx	83.73%448/535	75.45%83/110	62.5%5/8	83.73%448/535

Code coverage generated by Istanbul at 2026-10-09T22:10:04.173Z

Code coverage generated by Istanbul at 2026-10-09T22:10:04.173Z

The frontend's screens, file by file: the map (MapExplorer.tsx) and Profile.tsx are the least tested; the admin screens are in their own folder.

4. What isn't tested, and what to do next

Not covered	Instead
End-to-end tests in a real browser	Checked by hand in Chrome at phone and laptop sizes; no automated suite (such as Playwright) yet.
The map drawing and real GPS	jsdom can't lay out the map; checked by hand, and on campus for GPS.
The first request after the server sleeps	Render's documented behaviour; a keep-awake ping is the fix.
Load in CI	The load test is run by hand, against the live API.

Next steps, most useful first

- **Speed:** load screens only when they're opened (code splitting), so the first visit doesn't download the admin console and the map; load fonts without blocking the first paint.
- **Accessibility:** give the map's event markers a name and make them reachable by keyboard; enlarge the player marker's tap area; make the compass and display-name buttons' names match what they show.
- **SEO and best practices:** add a meta description; ship source maps; keep map text at 12 px or more.
- **Server:** keep the API awake during marking, and consider a region nearer South Africa.
- **Coverage:** tests for the map and admin screens, and for routes/telemetry.ts, the backend's largest gap.

Reproducing this report

```
cd scripts/performance && npm install
npm run load          # the load test          -> results/load-test.json
npm run lighthouse    # Lighthouse            -> results/lighthouse.json, reports/lighthouse.html
# the coverage runs (see scripts/performance/README.md) -> reports/coverage-*
npm run report        # this PDF               -> docs/reports/performance-and-coverage-report.pdf
```